

Student Activity Tracker for Education Institution

DATABASE CLASS PROJECT - FINAL REPORT

Date: 21 December 2025



GROUP 6: Muhammad Keenan Basyir (24/536808/PA/22775), Marvel Maliq Prabawa (24/532920/PA/22553), Fayyadh Dhia Abyan (24/546698/PA/23213)

I. INTRODUCTION

1.1. Problem Background

In many educational Institutions, the process of tracking student participation in extracurricular activities remains manual and fragmented (like sports, clubs, organizations, volunteer work). These records are often stored in a spreadsheet that is disparate, paper forms, or email threads. This decentralization leads to numbers of issues that are critical like:

- Data gets lost: It happens a few time data can get lost as when records are kept in different places, it is easy to lose a form or spell a name wrong. There is no single “master list” to trust.
- It wastes time: Teachers and staff spend hours typing data from paper forms into computers instead of actually helping the students
- Students also often struggle to remember exactly what did they do over the years in extracurricular activities. If they need a list of activities/organizations for a job application or resume, they have to piece it together from memory.

1.2. Project Objectives

We created a student activity tracker (called UniTracker) to solve these specific problems and our main goal for this project is to build a simple, web-based system that replaces the paper mess with a clean and efficient digital process.

For this project, we set out to achieve:

1. One Central Database: We wanted to store everything such as the activity records of students in one place that is secure so that the data is always safe and easy to find.
2. Digital Approvals: We wanted to get rid of physical signatures. Now, a student can submit a record online and the teacher can either click “Approve” or “Reject”.
3. Clean Data: We followed strict database rules or (3NF) to make sure that our data is correctly organized. This prevents duplicate entries and keeps the system running smoothly.
4. Student Record History: We aimed to provide students like a transcript of all their activities so that they can view what they have accomplished (the specific extracurricular activities).
5. Simple & Modern Design: We aimed to build the site with a professional look and easy enough for anyone to use.

1.3. System Users

For this project, our system is built for two specific groups of users:

1. Teachers & Faculty Staff:

These are the people that will be managing the records and running the system, like club advisors or faculty staff.

They are in charge of the data. They create the list of available extracurricular activities and manage the data records of them. They also act as the judges who look at incoming requests and decide they should be approved or rejected. They will need a dashboard that clearly shows them what work/requests is pending.

2. Students: The main users of the application:

The students use the site to track their own involvement. They log in to join to submit their work of their clubs/extracurriculars and record of hours in those organizations. They submit a request of it and gives them a clear way to see they activity records, hours spent in those clubs, and whether it is approved or rejected by the administrators (teachers or faculty staff).

1.4. Key Use Cases

In order to make the system work, we focused on these main actions:

- **Joining an Activity:** A student needs to be able to pick an activity or organizations from a list, type in their role and hours, and upload the request for review. The system then automatically marks this as "Pending."
- **Approving Records:** An admin logs in and sees a list of pending requests. They can review the details and click whether to approve to make it official or reject it.
- **Viewing Reports:** A student can check their record history at any time to show every activity they have ever done which acts as a digital portfolio of their time at school or university.

II. DATABASE DESIGN

2.1. Entity Relationship Diagram (ERD)

The foundation of our student activity tracker system is a relational database that is designed to link students, faculty, and activities together efficiently. We started creating a conceptual model (ERD) to visualize these relationships.

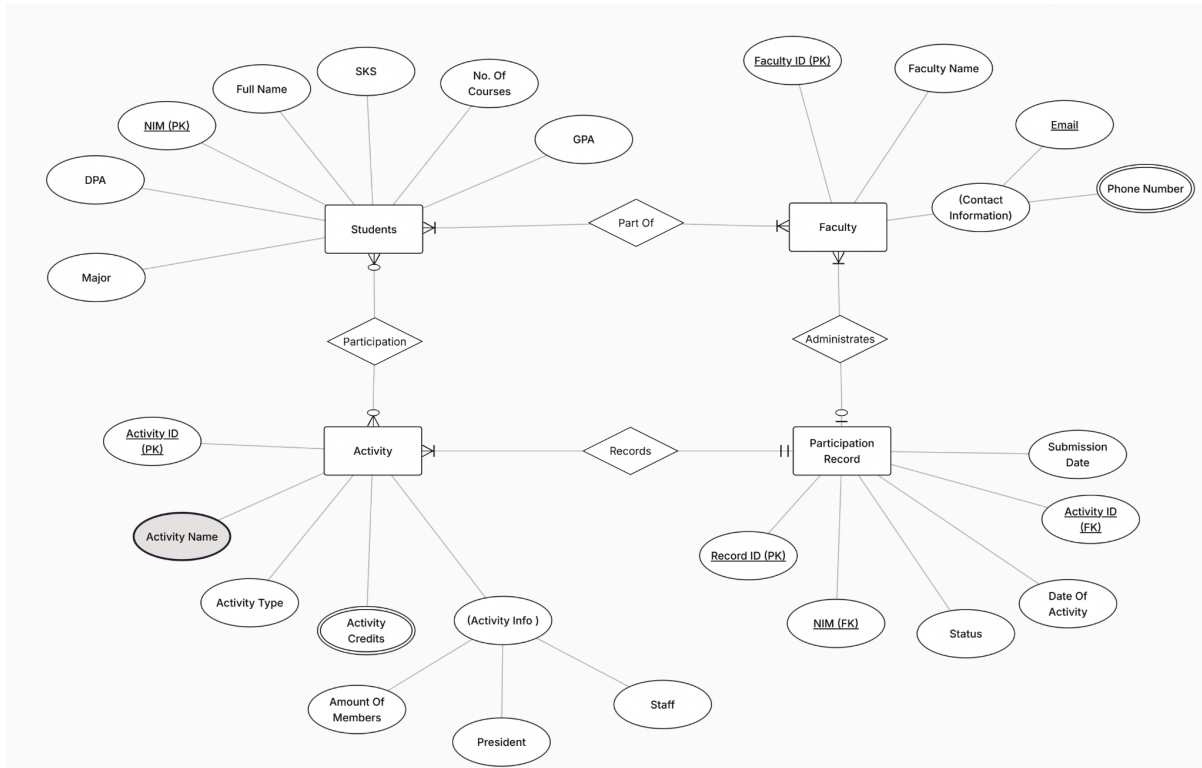


Figure 1: ERD of our student activity tracker system

2.2. Entity Explanations

For our student activity tracker system, our database consists of four main entities:

1. The Main Entities (The Rectangles)

- **Students:** Represent the students attending the educational Institution university.
- **Faculty:** This refers to the department or the academic body where the student is studying and belongs to.

- Activity: Stores information about the various events, clubs, or programs available, the amount of credits worth and a short description about the activity.
- Participation Record: Tracks the specific details of a student joining an activity.

2. Breakdown of the Attributes

Students:

- NIM (PK): The Student ID number. It is the primary key as it is the unique identifier for each student.
- Full name: Student's full name.
- SKS: Credits of academics.
- No. Of Courses: The number of courses taken by the students.
- GPA: Grade Point Average of students.
- Major: Specific field of study of what they study in university.
- DPA: Academic advisor.

Faculty:

- FacultyID (PK): The unique ID for the faculty department.
- Faculty Name: Name of faculty/department.
- Contact Information: This is a composite attribute which breaks down into smaller parts, Email which is the faculty's email address and Phone Number which is Multivalued Attribute meaning the faculty has more than one phone number listed.

Activity:

- Activity ID (PK): The unique ID for the extracurricular activities.
- Activity Name: The title of the activity.
- Activity Type: This is Multivalued Attribute, meaning an activity might offer different types or amounts of credits.
- Activity Info: Composite attribute containing Amount of Members, President (leader of the activity) and Staff (people running it).

Participation Record:

- Record ID (PK): A unique receipt number for the specific participation entry.
- Submission Date: The date the record was handed in.
- Date Of Activity: When the event actually happened.
- Status: The current state of the record being Pending, Approved or Rejected
- NIM (FK): A foreign key that connects this record back to a specific student
- Activity ID (FK): Another foreign key connecting this record back to a specific activity.

The Relationships

1. Students and Faculty: Many-to-Many (N:N)
 - The Connection: Part Of
 - There is a three branch line (crow's foot) on the Student side and on the Faculty side which implies a Many-to-Many relationship. A student can belong to more than one faculty, and a faculty obviously has many students.
2. Students and Activity: Many-to-Many (N:N)
 - The Connection: Participation
 - A crow's foot (three branch line) on Students and on Activity. This means it is a Many-to-Many relationship. A single student can join many different extracurricular activities, and a single activity consists of many different students.
3. Faculty and Participation Record: One-to-Many (1:N)
 - The Connection: Adminstrates
 - A crow's foot on Faculty side and single line on Record side. This is a One-to-Many relationship as one faculty department administrates many different Participation Records. But a specific Participation Record is managed by only one Faculty.
4. Activity and Participation Record: One-to-Many (1:N)
 - The Connection: Records
 - A crow's foot on Activity side and double vertical lines on the Record side. This is a One-to-Many relationship. One Activity will generate many different Participation Records (one for each student who attended).

2.3. Normalization Steps (3NF)

In order to make sure our database remains efficient, organized and free of errors, we applied the standard rules of Normalization up to the Third Normal Form or 3NF.

First Normal Form (1NF):

- For 1NF, we ensure that every single table cell holds only a single piece of data (rather a list of items).
- When analyzing our initial design, we noticed that the Faculty entity contained a “Phone Number” attribute inside a double circle. This meant a faculty could have multiple phone numbers. If we stuffed all the phone numbers into a single box separated by commas, it would be impossible to filter correctly.
- The Fix: We identified these “multivalued attributes” specifically Phone Number for Faculty and Activity Credits for Activities. We solved this problem by moving them out of the main tables and creating separate sub-tables. This would ensure each row contains only one valid number or credit value.

Second Normal Form (2NF):

- For 2NF, we ensure every column in a table depends on the entire primary key, especially when linking two different entities.
- In our Participation Record table, we are effectively linking a Student to an Activity. A common mistake would be to include the Student Name or Activity Type inside the record table. However, those details don’t depend on the specific record ID, they depend only on the Student ID or Activity ID.
- The Fix: We ensure that the Participation Record table stores data relevant to the event of participating like Submission Date and Status. We kept the static details (like names and majors) in their original Student and Activity tables. Therefore, if a student corrects a typo in their name, we don’t have to update thousands of participation records.

Third Normal Form (3NF):

- For 3NF which is the final step is to ensure that non-key attributes depend only on the primary key and not on other temporary data.

1. Calculated Data: The Activity entity originally included an attribute for *Amount Of Members*. Storing this as a hard number is risky because every time a new student joins, that number becomes outdated immediately.
 - The Fix: We treat this as a derived attribute. Instead of typing it into the table, the system should calculate it automatically by counting the rows in the Participation Record table. This guarantees the count is always 100% accurate without manual updates.
2. Activity Roles: The Activity entity also listed President and Staff. If we stored the text names of these students (e.g., "Joshua Rosfield"), it would violate 3NF because "Joshua Rosfield" depends on that person's identity, not the Activity itself.
 - The Fix: We structure this by referencing the student's unique NIM (Student ID) rather than typing their name. The database can then look up the Students table to find out who the president is. This prevents errors if the student changes their name or leaves the university.

III. DATABASE IMPLEMENTATION

(WE HAVE MADE A FEW CHANGES/IMPROVEMENTS OF OUR ERD DIAGRAM IN OUR RELATIONAL SCHEMA)

We implemented our database using SQLite.

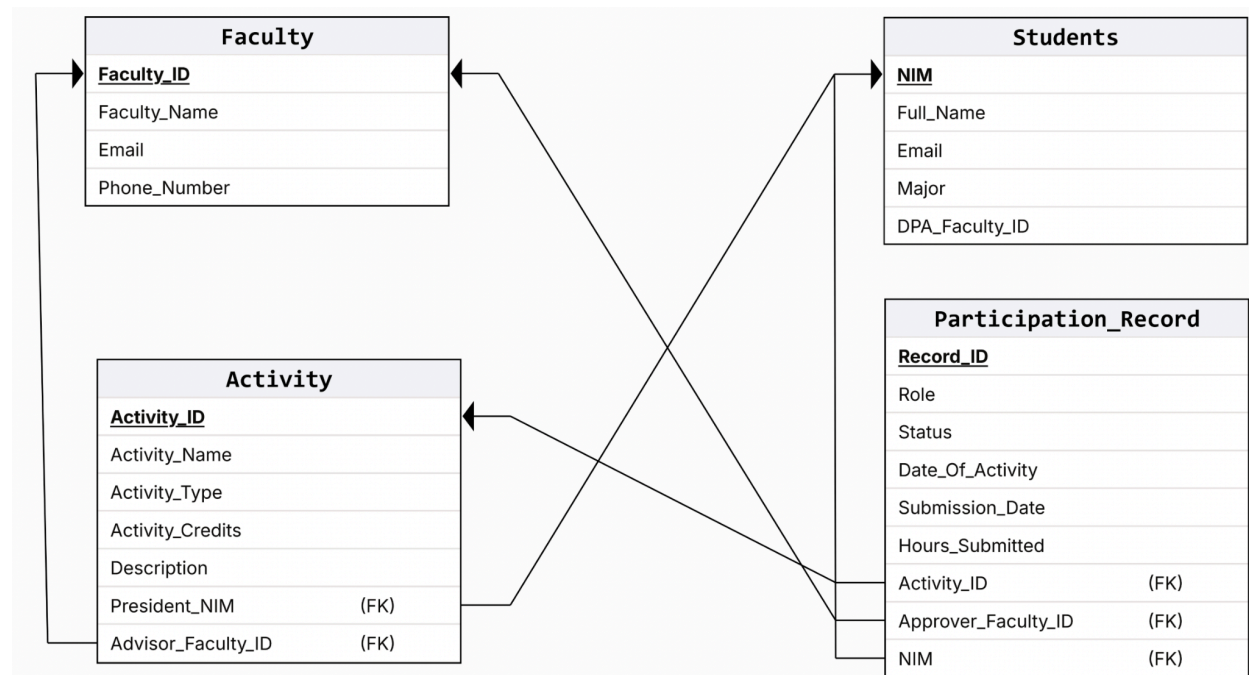


Figure 2: Relational Schema of our updated student activity tracker system

3.1. Improvements from ERD to Relational Schema

Feature	Original ERD	New Relational Schema	Improvement
Student Leader	President (Text)	President_NIM (Foreign Key)	Connects to real data and prevent typos. Instead of typing the name, we can link directly to the NIM in the Students table of the President (3NF fix).
Faculty Role	Adminstrates (Generic line/relationship)	Approver_Faculty_ID & Advisor_Faculty_ID	The specific roles are now clearly defined. This links an extracurricular activity/organizations to a specific Faculty overseer. The Approver_Faculty_ID tracks exactly who approved a specific student's request.
Student Data	DPA (Vague)	DPA_Faculty_ID	It clearly links a student to their advisor. This links a student to their Academic Advisor.

Additional Removals

Removed Attribute	From Table	Reason
GPA / SKS	Students	Derived data: Don't store what we can calculate. Prevents data mismatch
Amount of Members	Activity	Derived data: Don't store the count; just count the records when needed and ensures the count is always 100% accurate. For example if a new student joins, we have to update that number from "50" to "51".
Staff	Activity	Redundant/Messy: Replaced by the Role column in the Participation Record to define who is staff.

3.2. DDL Scripts

We implemented our database using SQLite. The following Data Definition Language (DDL) script was used to create the tables and that this script reflects our final 3NF design.

```
1  CREATE TABLE Faculty (
2      Faculty_ID INTEGER PRIMARY KEY AUTOINCREMENT,
3      Faculty_Name VARCHAR(100) NOT NULL,
4      Email VARCHAR(100) UNIQUE NOT NULL,
5      Phone_Number VARCHAR(20)
6  );
7
8  CREATE TABLE Students (
9      NIM INTEGER PRIMARY KEY,
10     Full_Name VARCHAR(100) NOT NULL,
11     Email VARCHAR(100) UNIQUE NOT NULL,
12     Major VARCHAR(50),
13     DPA_Faculty_ID INTEGER,
14     FOREIGN KEY (DPA_Faculty_ID) REFERENCES Faculty(Faculty_ID)
15 );
16
17 CREATE TABLE Activity (
18     Activity_ID INTEGER PRIMARY KEY AUTOINCREMENT,
19     Activity_Name VARCHAR(100) NOT NULL,
20     Activity_Type VARCHAR(50),
21     Activity_Credits INTEGER DEFAULT 0,
22     Description TEXT,
23     Advisor_Faculty_ID INTEGER,
24     President_NIM INTEGER,
25     FOREIGN KEY (Advisor_Faculty_ID) REFERENCES Faculty(Faculty_ID),
26     FOREIGN KEY (President_NIM) REFERENCES Students(NIM)
27 );
28
29 CREATE TABLE Participation_Record (
30     Record_ID INTEGER PRIMARY KEY AUTOINCREMENT,
31     NIM INTEGER NOT NULL,
32     Activity_ID INTEGER NOT NULL,
33
34     Role VARCHAR(100),
35     Status VARCHAR(20) NOT NULL DEFAULT 'Pending',
36     Date_Of_Activity DATE,
37     Submission_Date DATE NOT NULL,
38     Hours_Submitted INTEGER,
39
40     Approver_Faculty_ID INTEGER,
41
42     FOREIGN KEY (NIM) REFERENCES Students(NIM) ON DELETE CASCADE,
43     FOREIGN KEY (Activity_ID) REFERENCES Activity(Activity_ID) ON DELETE CASCADE,
44     FOREIGN KEY (Approver_Faculty_ID) REFERENCES Faculty(Faculty_ID)
45 );
```

3.3 Key Constraints

To maintain the integrity of our data, we applied several critical constraints to our tables:

1. Primary keys (PK): Every table has a unique identifier. For example NIM or Activity_ID ensures no two records are identical.
2. Foreign Keys (FK): We used Foreign Keys to enforce relationships like for example, Participation_Record must link to a valid Student and Activity. DELETE CASCADE was also used so that when a student is removed or delete, their participation history is automatically removed to prevent “orphan” records.
3. NOT NULL: This is the critical, important fields like Full_Name, Email, and Activity_Name which cannot be left empty.
4. UNIQUE: We enforced a UNIQUE constraint Email addresses in the Student and Faculty tables in order to prevent duplicate accounts.
5. DEFAULT Values: The Status field in Participation_Record defaults to 'Pending'. This simplifies the submission process for students.

3.4 Example SQL Statements (CRUD)

Our web application interacts with the database using SQL commands (containing CRUD operations).

1. Creating a New Student:

```
INSERT OR IGNORE INTO Students (NIM, Full_Name, Email, Major, Password) VALUES
(24536808, 'Muhammad Keenan Basyir', 'keenan@university.edu', 'Computer Science', 'password123'),
(24532920, 'Marvel Maliq Prabawa', 'marvel@university.edu', 'Information Systems', 'password123'),
(24546698, 'Fayyadh Dhia Aryan', 'fayyadh@university.edu', 'Computer Engineering', 'password123');
```

2. Creating/inserting administrators:

```
INSERT OR IGNORE INTO Faculty (Faculty_Name, Email, Phone_Number) VALUES
('Dr. Alice Johnson', 'alice@university.edu', '555-0101'),
('Prof. Bob Smith', 'bob@university.edu', '555-0102'),
('Dr. Carol White', 'carol@university.edu', '555-0103');
```

3. Creating/Inserting activities:

```
INSERT OR IGNORE INTO Activity (Activity_Name, Activity_Type, Activity_Credits, Description) VALUES  
( 'Badminton Club', 'Sports', 3, 'Weekly badminton practice.' ),  
( 'Robotics Workshop', 'Academic', 5, 'Building autonomous robots.' ),  
( 'Charity Run Volunteer', 'Volunteer', 2, 'Helping organize the run.' ),  
( 'Photography Club', 'Arts', 3, 'Visual storytelling workshop.' );
```

4. Admin Approval Dashboard (SELECT with JOIN):

```
1  SELECT  
2      pr.Record_ID,  
3      s.Full_Name AS Student_Name,  
4      a.Activity_Name,  
5      pr.Role,  
6      pr.Status,  
7      pr.Submission_Date  
8  FROM Participation_Record pr  
9  JOIN Students s ON pr.NIM = s.NIM  
10 JOIN Activity a ON pr.Activity_ID = a.Activity_ID  
11 ORDER BY pr.Submission_Date DESC;
```

5. Approving a Record (UPDATE):

```
UPDATE Participation_Record  
SET Status = 'Approved', Approver_Faculty_ID = 1  
WHERE Record_ID = 5;
```

6. Deleting a Student (DELETE):

```
DELETE FROM Students WHERE NIM = 1001;
```

IV. APPLICATION IMPLEMENTATION

3.1 TECHNOLOGY STACK & JUSTIFICATION

To build a robust, scalable, and user-friendly Student Activity Tracker, we carefully selected a technology stack that balances modern development practices with the specific requirements of a database-centric application.

Frontend: React.js

- React's component-based architecture allowed us to build reusable UI elements (like the ProfileCard and ActivityCard) that ensure visual consistency across the application. Its Virtual DOM provides a highly responsive user experience, which is critical for dynamic dashboards where data updates frequently. We utilized React Router for seamless client-side navigation between the Login, Student Dashboard, and Admin Dashboard views without triggering full page reloads.

Backend: Node.js & Express

- The server-side logic is built on Node.js using the Express framework. Node.js allows us to use JavaScript for both frontend and backend development, streamlining the coding process. Express provides a minimalist yet powerful interface for defining RESTful API endpoints (GET, POST) that handle data requests from the client. Its middleware architecture enabled us to easily integrate essential features like CORS (Cross-Origin Resource Sharing) for security and Multer for handling multipart file uploads.

Database: SQLite

- We implemented the relational database using SQLite3. For this project, SQLite offers an ideal balance of power and simplicity. Unlike client-server databases (like MySQL or PostgreSQL), SQLite is serverless and self-contained, storing the entire database as a single file (student_tracker.db). This makes the application highly portable and easy to deploy. Despite its simplicity, it fully supports the complex SQL queries, JOIN operations, and 3NF normalization required by the project specifications.

3.2 SYSTEM ARCHITECTURE

The application follows a classic Three-Tier Architecture, ensuring a clear separation of concerns between the user interface, business logic, and data storage.

A. Presentation Layer (Client):

The Presentation Layer handles all user interaction and visual rendering.

- Technology: React.js (Single Page Application)
- Core Mechanics:
 - SPA Architecture: The application loads a single HTML entry point. Subsequent navigation is handled by React Router, which intercepts URL changes and updates the view dynamically without triggering a full page reload. This results in faster transitions and a more "app-like" feel compared to traditional multi-page websites.
 - Component Composition: The UI is built from modular components (e.g., ProfileCard, ActivityCard, ModalPopup). These components are reused across different "views" to maintain design consistency and reduce code redundancy.
 - State Management: We utilize React Hooks (useState, useEffect) to manage local data states (like form inputs or loading indicators) and localStorage to persist user sessions across browser refreshes.
 - API Integration: The frontend communicates with the backend exclusively via asynchronous RESTful API calls (fetch), exchanging data in JSON format or FormData for file uploads.

B. Application Layer (Server):

The Application Layer serves as the bridge between the user and the database. It processes business rules, validates input, and handles security.

- Technology: Node.js with Express Framework
- Core Responsibilities:
 - API Gateway: The Express server exposes a set of defined endpoints (e.g., POST /api/login, GET /api/activities). Each endpoint acts as a specific function call that the frontend can trigger.
 - Request Processing Pipeline: Incoming requests pass through middleware layers. CORS middleware ensures security by allowing requests only from authorized domains (our frontend). JSON Body Parser converts raw request data into usable JavaScript objects.
 - Business Logic Execution: This layer contains the core rules of the application. For example, the "Review Request" logic ensures that only a user with the Faculty role can

update a record's status. It effectively guards the database from invalid or unauthorized changes.

- File Handling Service: We implemented a dedicated file handling service using Multer. When a student uploads a proof file, this service intercepts the stream, assigns a unique filename (timestamp + original name) to prevent overwrites, saves it to the local /uploads directory, and passes the file path to the database logic.

C. Data Layer (Database):

The Data Layer is responsible for the reliable storage and retrieval of organized information.

- Technology: SQLite3
- Data Structure: The database follows a relational model normalized to the Third Normal Form (3NF).
 - Entities: Core entities include Students, Faculty, Activity, and Participation_Record.
 - Relationships: Foreign keys define strict relationships. A Participation_Record cannot exist without linking to a valid Student (via NIM) and a valid Activity (via Activity_ID).
- Database Operations:
 - Initialization & Migration: We implemented a custom initialization script in server.js. On startup, it reads the StudentActivityTracker.sql schema file to build the database structure if it doesn't exist. It also performs smart schema updates, such as checking for the existence of new columns (e.g., Proof_File) and adding them dynamically via ALTER TABLE commands. This ensures the database schema stays in sync with the application code without manual intervention.
 - Query Execution: The application uses parameterized SQL queries (e.g., WHERE Email = ?) to prevent SQL Injection attacks, ensuring that user input is treated as data, not executable code.

3.3 UI/UX DESIGN

The design philosophy for the Student Activity Tracker focused on Clarity, Efficiency, and Modern Aesthetics. Moving away from the traditional, text-heavy database interfaces to create a tool that feels like a modern web app.

Visual Theme: "Electric Indigo"

- We adopted a Dark Mode foundation (#1a1a1a) to reduce eye strain and provide a sleek, professional canvas.
- To create contrast and visual hierarchy, we used Electric Indigo (#4f46e5) as our primary accent color. This vibrant hue is used exclusively for interactive elements (buttons, active tabs, focus states), intuitively guiding the user's attention to key actions.

UX Optimizations

- **Modal Popups:** Instead of navigating to a separate page to view activity details, we implemented a Modal Popup. Clicking an activity card opens a focused overlay with details (Credits, Advisor, Description). This keeps the user in the context of the dashboard and makes browsing faster.
- **Role-Based Dashboards:** The UI dynamically adapts based on the user's role. Students see a "Submission" interface, while Faculty see an "Approval Queue." This minimizes cognitive load by only showing features relevant to the current user.
- **Immediate Feedback:** Interactive elements like buttons feature smooth transform animations on hover, providing tactile feedback that the application is responsive and ready for input.

3.4 MAIN MODULES

1. Authentication Module

- **Purpose:** Manages user access and security.
- **Key Functions:**
 - Verifies credentials (email/password) against the Students and Faculty database tables.
 - Determines user roles (Student vs. Faculty) upon successful login.
 - Manages session state (using localStorage) to keep users logged in.
 - Handles redirection to the appropriate dashboard based on the user role.

2. Student Dashboard Module

- **Purpose:** The central hub for student interactions.
- **Key Components:**
 - **Profile Viewer:** Fetches and displays student details (Name, ID, Major, Status).
 - **Activity Browser:** dynamically lists available extracurricular activities fetched from the database (Activity table). It includes a detailed modal/popup view for each activity showing credits, descriptions, and advisors.

- History Tracker: Displays a personal list of participation records with real-time status updates (Pending, Approved, Rejected).

3. Submission & File Handling Module

- Purpose: Facilitates the creation of new participation records.
- Key Functions:
 - Data Collection: A form interface for selecting activities, dates, roles, and hours.
 - File Upload: Handles the upload of "Proof of Participation" (PDFs).
 - Data Processing: Uses FormData on the frontend and multer middleware on the backend to process multipart requests.
 - Storage: Saves file metadata to the database and the actual file to the server's filesystem.

4. Administrative Module (Faculty Dashboard)

- Purpose: Provides oversight and approval capabilities for faculty.
- Key Functions:
 - Pending Queue: Fetches and displays all records with a "Pending" status.
 - Review Interface: Allows admins to view details and access the uploaded proof file via a secure link.
 - Decision Logic: Updates the record status in the database to "Approved" or "Rejected" based on the admin's action.

5. Database Interaction Module (Backend API)

- Purpose: Serves as the bridge between the frontend and the database.
- Key Functions:
 - Exposes RESTful endpoints (GET, POST) for data retrieval and manipulation.
 - Executes SQL queries (SELECT with JOINS, INSERT, UPDATE) to maintain data integrity.
 - Manages database initialization and schema migration (auto-creating tables and columns).

V. TESTING AND RESULTS

5.1 LOGIN PAGE TESTING

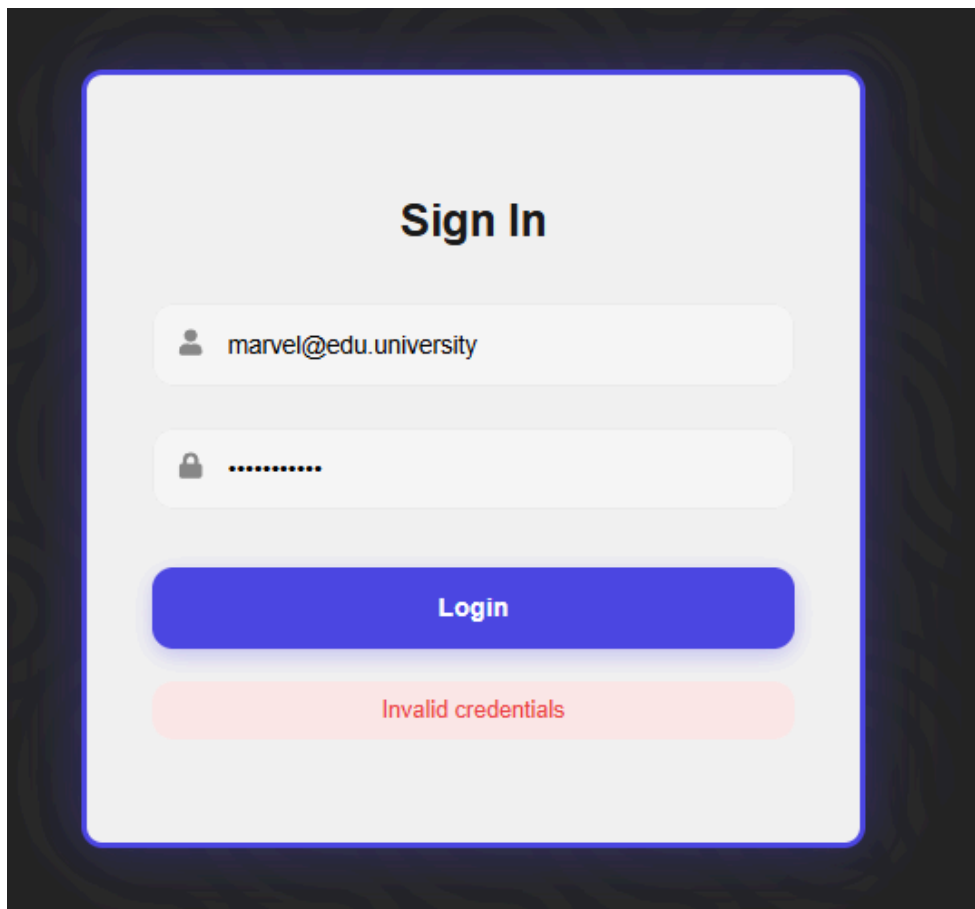
LOGIN ACCOUNTS TESTING

BOB@UNIVERSITY.EDU PASS: ADMIN123

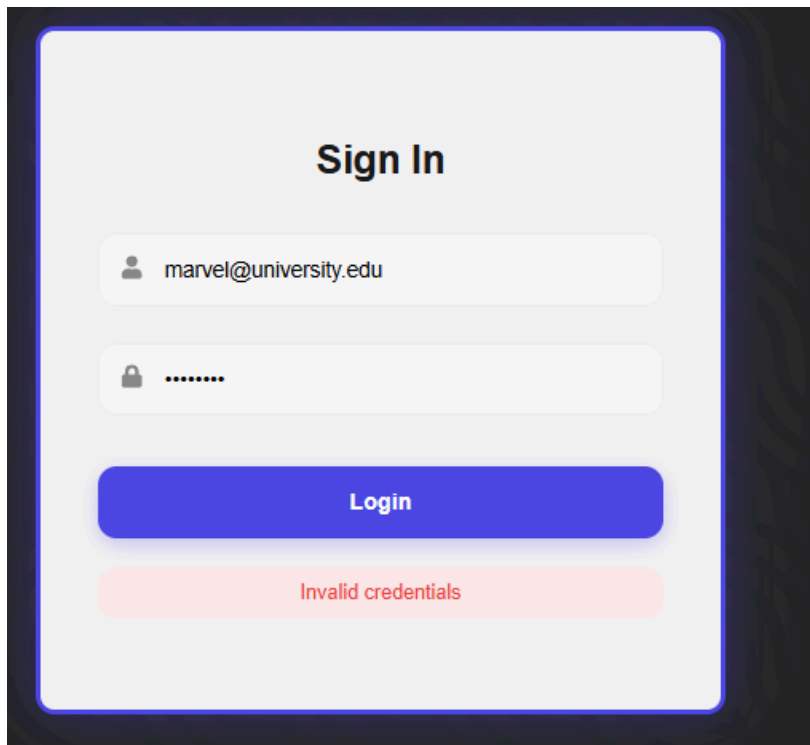
MARVEL@UNIVERSITY.EDU PASS:PASSWORD123

TESTING OUTPUT OF LOGIN USING BOTH INCORRECT CREDENTIALS AND PASSWORD TO TEST BASIC SECURITY

IN THIS FIRST ATTEMPT USING THE WRONG CREDENTIALS I WILL TRY TO LOG INTO AND ACCESS THE MAINPAGE, AS SEEN IN THE IMAGE BELOW THE OUTPUT IS THAT ACCESS IS NOT GRANTED AND A MESSAGE SAYING THAT INVALID CREDENTIALS



SECOND ATTEMPT USING INCORRECT PASSWORD (ADMIN123), SAME RESULTS AS BEFORE UNABLE TO ACCESS THE MAIN PAGE AND OUTPUTS AN INVALID CREDENTIALS

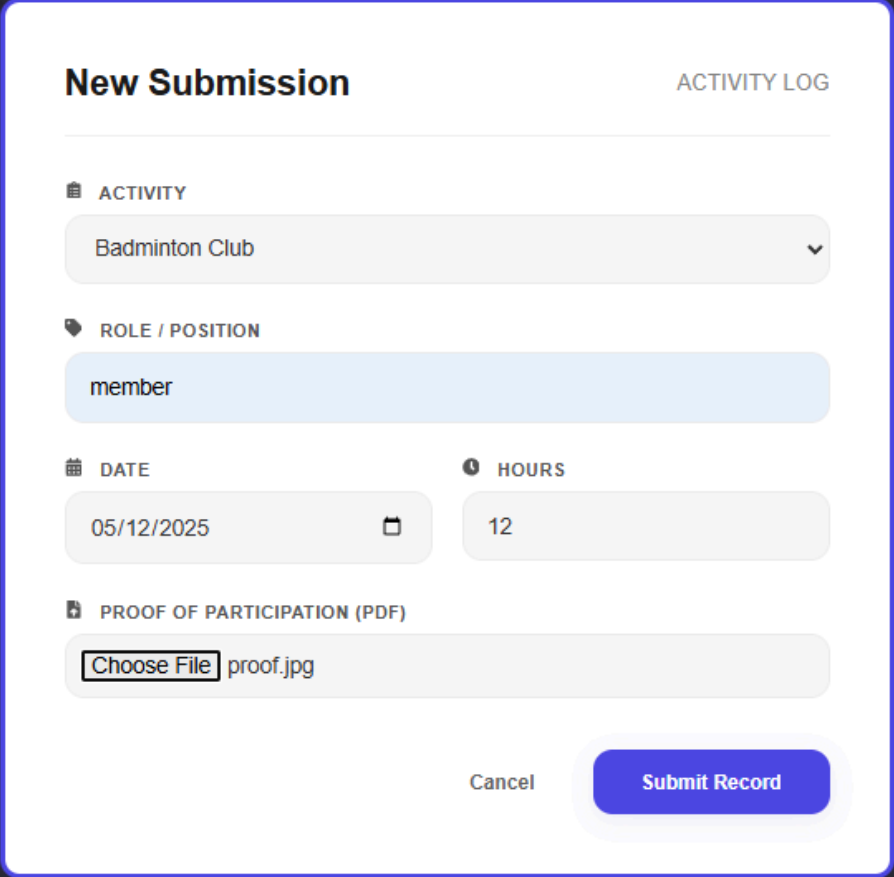


Sign In

Login

Invalid credentials

ONCE IN THE RECORDS AREA THEY CLICK A PLUS SIGN AND WILL BE TAKEN TO THE SUBMISSION TABLE, IN THE TABLE WILL INCLUDE INFORMATION SUCH AS WHAT THE ACTIVITY IS THEIR ROLE WHEN THEY HAVE JOINED AND HOW MANY HOURS THEY HAVE INVESTED INTO THE ACTIVITY, IN THE BOTTOM THERE WILL ALSO BE A PARTICIPATION PROOF WITH AN IMAGE UPLOADER.



The image shows a 'New Submission' form with a blue border. At the top left is the title 'New Submission' and at the top right is a link 'ACTIVITY LOG'. The form contains several input fields: a dropdown menu for 'ACTIVITY' with 'Badminton Club' selected, a text field for 'ROLE / POSITION' with 'member' entered, a date picker for 'DATE' showing '05/12/2025', and a text field for 'HOURS' showing '12'. Below these is a file upload section for 'PROOF OF PARTICIPATION (PDF)' with a 'Choose File' button and the filename 'proof.jpg'. At the bottom right are two buttons: 'Cancel' and 'Submit Record'.

New Submission [ACTIVITY LOG](#)

ACTIVITY
Badminton Club

ROLE / POSITION
member

DATE 05/12/2025 **HOURS** 12

PROOF OF PARTICIPATION (PDF)
Choose File proof.jpg

Cancel **Submit Record**

IN THE ADMIN MAIN PAGE IT WILL SHOW HOW MANY REQUESTS THERE ARE AND THE REQUEST ITSELF, ADMINS CAN GO OVER THE REQUEST AND EITHER APPROVE OR DENY IT DEPENDING ON THE PROOF .



FACULTY ADMINISTRATOR

PROF. BOB SMITH

bob@university.edu

ROLE

Approver

DEPARTMENT

Academic Affairs

PENDING

2 Requests

PENDING STUDENT REQUESTS

Refresh

Badminton Club

Student: Marvel Maliq Prabawa

Sports

ROLE

DATE

HOURS

member

2025-12-05

12 hrs

View Proof of Participation

Approve

Decline

Robotics Workshop

Student: Marvel Maliq Prabawa

Academic

ROLE

DATE

HOURS

member

2025-12-10

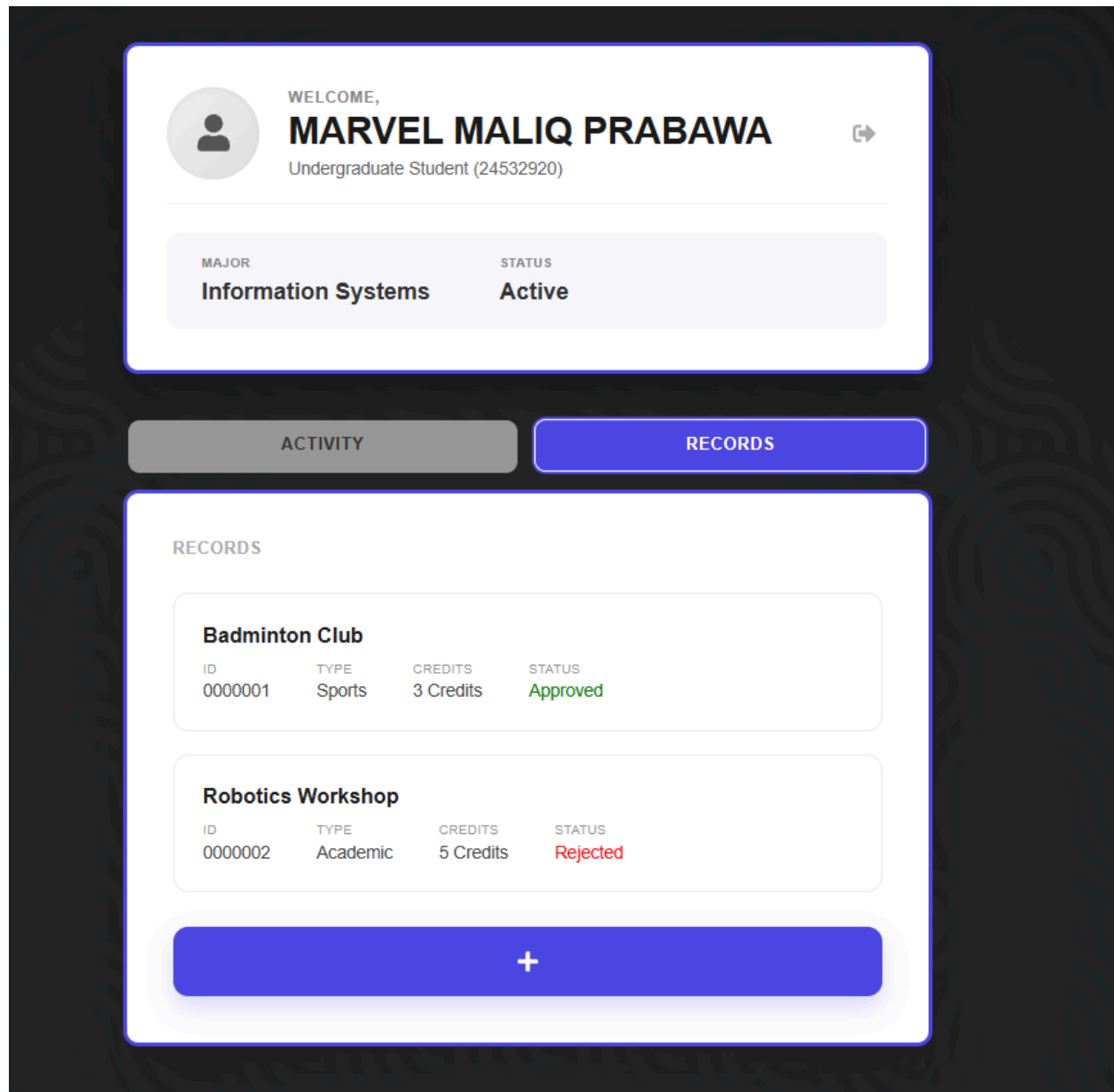
22 hrs

View Proof of Participation

Approve

Decline

ONCE APPROVED OR DENIED BY THE ADMIN THE USER WILL GO TO THE RECORD TAB AND SEE IF IT WAS APPROVED OR DENIED WITH THE STATUS PART.



The image shows a user interface for a student profile and their records. The background is dark with a subtle pattern. The main content is in a white box with a blue border. At the top, there's a welcome message and the user's name, Marvel Maliq Prabawa, with a share icon. Below this, the user's role is listed as Undergraduate Student (24532920). A table shows the user's major as Information Systems and their status as Active. Below this, there are two tabs: Activity and Records. The Records tab is selected, showing a list of records. The first record is for the Badminton Club, with ID 0000001, Type Sports, 3 Credits, and Status Approved. The second record is for the Robotics Workshop, with ID 0000002, Type Academic, 5 Credits, and Status Rejected. At the bottom, there is a blue button with a white plus sign.

WELCOME,
MARVEL MALIQ PRABAWA 

Undergraduate Student (24532920)

MAJOR	STATUS
Information Systems	Active

ACTIVITY RECORDS

Badminton Club

ID	TYPE	CREDITS	STATUS
0000001	Sports	3 Credits	Approved

Robotics Workshop

ID	TYPE	CREDITS	STATUS
0000002	Academic	5 Credits	Rejected



VI. CONCLUSION AND REFLECTION

The primary objective of this project was to design and implement the Student Activity Tracker, a centralized web-based system aimed at resolving the inefficiencies of manual extracurricular record-keeping. We have successfully delivered a fully functional application that meets all core requirements outlined in our initial proposal.

5.1 SUMMARY OF ACHIEVEMENTS

- **Role-Based Access Control:** The system successfully differentiates between "Student" and "Faculty" users, providing tailored dashboards and permissions for each role. Students can browse activities and submit records, while faculty have exclusive access to administrative approval tools.
- **Automated Approval System:** We streamlined the validation process by implementing a real-time status tracking system. Students receive immediate feedback on their submissions (Pending, Approved, Rejected), eliminating the need for manual follow-ups.
- **Robust Technical Implementation:** The application demonstrates a seamless integration of a React.js frontend with a Node.js/Express backend and an SQLite database. We successfully tackled complex technical challenges, such as implementing secure file uploads for proof of participation and creating a self-healing database schema that auto-migrates on startup.
- **Modern User Experience:** The final product features a polished, responsive user interface with a "dark mode" aesthetic and glassmorphism elements, ensuring a professional and engaging user experience across devices.

The Student Activity Tracker stands as a comprehensive solution that modernizes student engagement management, reducing administrative overhead and empowering students to take ownership of their co-curricular portfolio.

5.2 TECHNICAL REFLECTION

To be able to integrate a fully functional full-stack website application, one of the most critical takeaways was understanding how to connect a React frontend with a Node.js backend. To achieve that, we learned how to structure API endpoints, manage asynchronous data flow using `fetch` and `useEffect`, and handle state updates to reflect database changes in real-time.

Furthermore, implementing a relational database from scratch for database management taught us the importance of normalization (3NF) and data integrity. Issues and challenges with duplicate data entries, were resolved by applying UNIQUE constraints and refining our SQL schema. Additionally, creating the auto-migration script was a major milestone, allowing us to evolve our database schema without losing existing data.

Finally, a significant technical hurdle was attempting to implement the file upload feature. This required moving from simple JSON data to multipart/form-data that required integrating FormData on the client side and multer middleware on the server. Overcoming this complexity gave us a deeper understanding of HTTP request structures and file system operations.

5.3 LIMITATION & FUTURE WORK

While the current system is fully functional, there are areas identified for future improvement:

- **Security:** Currently, passwords are stored in plain text for demonstration purposes. In a production environment, implementing password hashing and JSON Web Token authentication would be essential to secure user data.
- **Scalability:** The application currently relies on a local SQLite database file. As the user base grows, migrating to a more robust client-server database system like PostgreSQL or MySQL would offer better concurrency and performance.
- **Deployment:** The application runs on a local development server. Future work involves deploying the frontend and backend to cloud platforms (like Vercel, Heroku, or AWS) to make it accessible to users over the internet.
- **Notification System:** Adding an automated email notification system would improve user engagement, alerting students immediately when their submission status changes.
- **Reporting:** Developing a feature to generate and download comprehensive PDF reports of approved activities would add significant value for students building their portfolios.

We hope that the experiences gained from developing this project will assist us in developing and innovating greater projects in the future.

5.4 CLOSING STATEMENT

From a technical perspective, this project successfully integrated a modern technology stack utilizing React.js, Node.js, and SQLite within a robust Three-Tier Architecture. We overcame challenges related to file handling, database synchronization, and user interface design to deliver a polished, functional

product. While there is room for future enhancements such as advanced security and reporting features, the current implementation fulfills all primary objectives and provides a solid foundation for further development. This project demonstrates the tangible value of applying software engineering principles to solve real-world administrative problems.